

Unit 02: Classic C Compilers

Fall in Cleveland. The leaves are turning. The academic year is beginning, and you are about to read a compiler.

What this unit is

A compiler is a program that reads a program and produces another program. That sentence sounds circular until you realize it is. The C compiler reads C and produces machine code. The C compiler is itself written in C. The first C compiler was not — it was written in B, and before that there was no C.

This unit is about that lineage. Not the theory of compilation — the grammars, the automata, the optimization passes — but the actual programs, the ones that existed and ran and made Unix possible. We are reading compilers the way we read novels: for what they reveal about the people who wrote them and the world they were written in.

You do not need to understand how a compiler works to begin reading one. You need to be curious about it.

The repos

Two compilers are installed in this unit.

`gcc/` — The GNU C Compiler. Richard Stallman started this in 1987 as the flagship of the GNU project. It is large — millions of lines across its history — and it is deeply opinionated. The opinionating is visible in the code, in the comments, in the architecture decisions, in the license. GCC is a political act as much as an engineering one.

`tcc/` — Fabrice Bellard's Tiny C Compiler. Roughly 82,000 lines. It compiles itself in seconds. Bellard wrote the core over a weekend as a contest entry, then kept going. It is a demonstration that elegance and capability are not in opposition — that a compiler does not have to be large to be real.

Start with `tcc`. It is small enough that the architecture is visible from any vantage point. Then move to `gcc`, with different expectations.

What is not here

Two compilers deserve mention even though they are not in the repository.

DMR's original C compiler (1972) — Dennis Ritchie's first C compiler, written initially in B and then rewritten in C itself. It lives in the Unix V6 source, available through the Unix Heritage Society at tuhs.org. At roughly 4,000 lines, it is the original. Everything in this directory descends from it, directly or spiritually. It is worth finding.

PCC — The Portable C Compiler (1977) — Steve Johnson's rewrite, designed explicitly for portability across processor architectures. For twenty years PCC was the standard Unix system compiler. OpenBSD maintained a version and it is still available. Johnson also wrote yacc, still in use. His work is less famous than it should be.

Both of these ancestors are readable. Finding and reading them is optional for this unit and strongly recommended for life.

Reading order

Start with `tcc/`.

Open `tcc.c`. The entire front end is there — parser, code generator, and the glue between them — in a single file. Find `main()`. Follow what happens when `tcc` reads a C source file. You will be able to see the whole architecture in an afternoon.

Then read one of the back ends: `i386-gen.c` or `x86_64-gen.c`. This is where C becomes machine instructions. Watch an abstract operation — an addition, a function call, a dereference — become actual bytes.

Then move to `gcc/`.

Do not try to read all of GCC. Nobody reads all of GCC. Instead: find `gcc.c` — the driver, the top of the call stack — and follow what happens when you type `gcc hello.c`. Read the architecture documentation. Find one optimization pass and understand what problem it solves.

The goal is not to understand every pass. The goal is to know what kind of thing a compiler is — to stand inside one the way you stood inside xv6.

Freely available reading

The following materials are free and worth having.

Primary

"The Development of the C Language" — Dennis M. Ritchie (1993). A Bell Labs technical report, freely available online. The author's own account of how C was designed and why. Start here before you start the code. Search the title; it is widely mirrored.

"Reflections on Trusting Trust" — Ken Thompson (1984). *Communications of the ACM*, vol. 27, no. 8. Eight pages. Thompson's Turing Award lecture. It asks: what does it mean to trust a compiler? The answer will change how you think about every piece of software you use. Widely reprinted and freely available.

The C Reference Manual — Dennis Ritchie. Included in the Unix Seventh Edition manual, available through tuhs.org. The first formal specification of C. Short, precise, historical.

Archives and Oral History

Unix Heritage Society — tuhs.org — V1 through V7 Unix sources, including early C compilers and their associated documentation. Free, searchable, invaluable for understanding where this all started.

Computer History Museum oral histories — computerhistory.org — video interviews with Ritchie, Thompson, Kernighan, Joy, and others who were there. Free. The oral histories are often more revealing than the written accounts.

GNU Compiler Collection Internals Manual — gnu.org/software/gcc/ — technical documentation for GCC's architecture and passes. Free, dense, authoritative.

Standard references (not free, but canonical)

The C Programming Language — Kernighan and Ritchie, 2nd ed., 1988. The K&R. If you have one book on C, this is it. Brief, clear, complete.

UNIX: A History and a Memoir — Brian Kernighan, 2019. Not expensive. Kernighan was at Bell Labs throughout the Unix and C years. He remembers what it was like, and he writes clearly.

Compilers: Principles, Techniques, and Tools — Aho, Lam, Sethi, Ullman. The Dragon Book. Theory. Not required for this unit, but you will eventually want it.

Engineering a Compiler — Cooper and Torczon. More approachable than the Dragon Book. A cleaner exposition of the same material.

The growth mindset note

You are not expected to understand how a compiler works before reading this unit. You are expected to be curious about how a compiler works. Those are different things, and the difference matters.

Curiosity gets you into the file. Understanding arrives later, sometimes much later, sometimes in a flash when you are doing something else entirely. Both are valid. Neither can be forced.

The goal of this unit is bearings, not mastery. Stand inside the code. Find one function you can follow end to end. Notice the architecture. Notice the patterns. Notice what you don't yet know — and notice that you can name what you don't yet know, which is itself a form of progress.

Welcome to Unit 02.

Next: open 02-ontogeny.pdf for the lineage — how this particular family of compilers came to exist. Then open tcc/tcc.c and find main().